EnterpriseSynth: Zero-Execution SFT and Eval Data from OpenAPI Specs

Anonymous submission

Abstract

Tool-using large language model agents require substantial training and evaluation data, yet collecting execution traces from enterprise APIs is often unsafe, expensive, or infeasible because of access controls, compliance constraints, rate limits, and the risk of modifying production data. We present EnterpriseSynth, a zero-execution framework that converts OpenAPI specifications into supervised fine-tuning trajectories and paired evaluation instances without invoking the underlying APIs. The framework uses a four-stage pipeline for specification parsing, graph-based dependency analysis, trajectory synthesis, and static validation. Each generated trajectory is checked against the source specification for endpoint, parameter, type, and schema consistency before being admitted to the dataset. We evaluate EnterpriseSynth on 17 real and synthetic APIs. Adversarial refinement increases the validator's detection rate for planted schema violations from 57–80% to 100%. Across five random seeds, a compact open-weight model fine-tuned on EnterpriseSynth data outperforms both an untuned model and a Self-Instruct baseline on held-out APIs. Performance remains consistent on five private, hand-authored API specifications that were not available during training, with tool-selection accuracy of 40.0% on the private set and 39.6% on the public held-out set. However, an independent LLM-based evaluation shows that endpoint-selection accuracy substantially overestimates end-to-end correctness: accuracy falls from 48.9% under the endpoint-level metric to 21.3% when argument selection and values are also evaluated. These results demonstrate that OpenAPI specifications can support scalable, execution-free data generation, while highlighting the need for argument-level evaluation when assessing the deployment readiness of tool-using agents.

1 Introduction

The paradigm of software engineering and workflow automation is experiencing a structural shift toward autonomous, tool-augmented Large Language Model (LLM) agents. By translating non-deterministic natural language intents into deterministic sequences of external system transactions—modeled as structured web interfaces via OpenAPI or Swagger specifications—agents possess the theoretical capacity to orchestrate highly complex enterprise operations.

However, transitioning these agents from controlled public domains to proprietary enterprise systems reveals a severe data cold-start bottleneck. Out-of-the-box foundation models exhibit poor zero-shot performance on enterprise tool configurations; they frequently violate JSON payload definitions, hallucinate non-existent parameter fields, and fail to track cascading variables across multi-step execution paths. Mitigating these structural failures necessitates robust Supervised Fine-Tuning (SFT) alongside highly localized evaluation tracks.

To populate these training datasets, state-of-the-art synthetic data generation engines rely exclusively on an active execution paradigm. Frameworks such as ToolBench [5] connect target foundation models to live execution servers or populated network backends, recording real-world system responses to construct functional interaction traces. While viable for public web extensions, this operational assumption collapses behind the enterprise firewall due to a threefold conflict termed here the *Execution Paradox*:

1. **Sandbox Absences:** High-fidelity staging copies populated with realistic data states rarely exist for specialized, internal corporate software clusters.
2. **Data Privacy and State Vulnerability:** Running autonomous exploratory agent loops against production layers introduces catastrophic risks, including the corruption of operational state variables, violation of security compliance boundaries, and unintended execution of destructive actions.
3. **Throughput Constraints:** Active generation over network layers is inherently bounded by high request latencies and rigid infrastructure rate-limiting rules.

To resolve this paradox, this paper presents **EnterpriseSynth**, a zero-execution synthesis framework that completely decouples agent data generation from system execution, shifting the data-gathering pipeline from a runtime extraction challenge to a static compiler-style validation problem. Given an enterprise API schema, EnterpriseSynth maps endpoints into candidate tool-use trajectories, uses an offline inference driver to synthesize textual reasoning paths, and enforces data-typing boundaries through an automated static constraint validation firewall.

The closest existing architecture to this offline synthesis approach is AgentInstruct [3], whose agentic multi-flow pipeline generates tool-use training data without executing any tool, simulating responses via the LLM itself. However, when seeded only from source code, AgentInstruct's

content-transformation stage must synthesize an API description from the code or have the LLM hypothesize additional APIs it believes exist, with no ground-truth check against a real interface definition; its quality control is a soft Suggester-Editor refinement loop plus a held-out, post-hoc LLM-judged benchmark, not a per-sample structural gate. EnterpriseSynth adapts its agentic-flow architecture to ingest a real target-organization OpenAPI/Swagger spec directly, removing the hallucination step entirely since the schema is already ground truth, replaces its post-hoc judging with a hard static constraint validator checked against the spec, and extends the pipeline to jointly emit intent-spec-tied evaluation records. Against the two execution-dependent alternatives reviewed below, API-Bank [2] and ToolBench [5], the distinction is safety and availability: both ground their training data in real API calls (API-Bank against reproducibility-constrained real databases, ToolBench via roughly 470,000 live RapidAPI calls during annotation), which is exactly the dependency the Execution Paradox rules out for most enterprise API surfaces.

1.1 Summary of Contributions

The contributions are scoped to what is actually implemented and measured: a zero-execution pipeline (schema parsing, intent synthesis, trajectory generation, and schema verification) that produces multi-turn instruction traces from real OpenAPI specs with no live backend connection; a deterministic, non-LLM static constraint validator, shown by adversarial testing to reach 100% detection of planted schema violations; and evidence that fine-tuning a small open model on the resulting verified trajectories improves tool-selection accuracy on genuinely held-out APIs. A dependency-graph-based planning layer is part of the target architecture (Section 3) but is not implemented, and no claim in this paper depends on it. The jointly-emitted evaluation dataset is released as **EnterpriseSynth-Eval**, named to avoid a collision with the unrelated, identically-named live-sandbox benchmark of Vishwakarma et al. [8]. The Limitations section below gives the full accounting of what is and is not built, and the Conclusion restates these contributions against the results that support them.

2 Related Work

This review covers two lineages – general-purpose synthetic instruction generation, and tool/API data generation – focusing on what each does or does not share with EnterpriseSynth, rather than each method's full mechanics.

2.1 Synthetic Instruction Generation

Self-Instruct [9] and **Evol-Instruct/WizardLM** [10] established that cheap, execution-free, heuristically-filtered synthetic data can scale instruction tuning, but neither touches tool-use or API grounding at all: both generate from free text, with no notion of a schema to check against. **AgentInstruct** [3] is the closest architectural precedent: an agentic multi-flow pipeline that produced the 25-million-pair dataset behind Orca-3, including a skill category. The two properties that distinguish EnterpriseSynth from it are structural. First, grounding: when AgentInstruct is seeded only from code, a transformation agent *synthesizes* an API description from it, or the LLM itself *hypothesizes* additional APIs it believes exist, with no ground-truth check – EnterpriseSynth ingests the real spec directly, so there is nothing to hallucinate. Second, verification: AgentInstruct's quality control is a soft Suggester-Editor refinement loop plus a held-out, GPT-4-judged benchmark (Orca-Bench) computed *after* generation, not a per-sample structural filter; EnterpriseSynth's verification is a hard, per-sample gate checked directly against the schema.

2.2 Execution-Dependent Tool-Use Data

API-Bank [2] and **ToolLLM/ToolBench** [5] both demonstrate that agentic pipelines can synthesize tool-use training data at scale (API-Bank's 1,888 dialogues at 98% lower cost than human annotation; ToolBench's 16,464 real endpoints and roughly 470,000 live RapidAPI calls logged during annotation). Both are effective precisely because they ground responses in real execution – which is exactly the capability that collapses behind an enterprise firewall (Section 1's Execution Paradox): no sandbox for internal systems, security/PII exposure from live calls, and infrastructure rate limits.

2.3 Inference-Time Tool-Use Methods

A separate line of work targets tool use at inference time rather than training-data synthesis. **Toolformer** [6] lets a model self-supervise *when* and *how* to call a small fixed set of tools by inserting API calls into its own generations and keeping only the calls that improve next-token prediction. **ReAct** [11] interleaves reasoning traces with actions in a single prompting loop, and **Reflexion** [7] adds verbal self-critique across repeated attempts to improve task success without any weight update. **Gorilla** [4] is closer in spirit to EnterpriseSynth's target task: it fine-tunes an LLM to generate correct, hallucination-free API calls against a large corpus of real API documentation. None of these four targets the enterprise cold-start setting – each assumes the API (or its documentation corpus) is already public, stable, and available for retrieval or live invocation, and none offers a deterministic, per-sample verification gate checked against a private spec. They are largely orthogonal to EnterpriseSynth rather than competing with it: these methods improve how a model uses tools at inference or through self-supervision on an existing API surface, while EnterpriseSynth addresses how to generate verified *training* data for tool use when no execution history or public documentation exists at all.

2.4 Research Gap

No existing framework combines the three properties above: grounding in a real target spec rather than text or code (unlike Self-Instruct, WizardLM, and AgentInstruct when seeded only from code), a hard per-sample verification gate rather than a soft or post-hoc one (unlike AgentInstruct), and no dependency on live execution (unlike API-Bank and ToolBench). This gap motivates EnterpriseSynth, a schema-aware framework that synthesizes user intents, reasoning

traces, API call sequences, and verification metadata directly from an OpenAPI specification, enabling enterprises to bootstrap both agent training and evaluation from API documentation alone.

3 Methodology

3.1 Implemented System

What is actually built and measured (Sections 6–8) is a four-stage pipeline:

$$\text{OpenAPI Spec} \rightarrow \text{API Schema Parser} \rightarrow \text{Intent Generation Agent} \rightarrow \text{Trajectory Generation Agent} \rightarrow \text{Schema-Aware Verification Engine} \rightarrow \{\text{SFT Dataset}, \textbf{EnterpriseSynth-Eval}\}$$

The jointly-emitted evaluation dataset is called **EnterpriseSynth-Eval** throughout this paper – named to avoid a collision with Vishwakarma et al.'s unrelated EnterpriseBench [8], a live-sandbox enterprise agent benchmark with a different scope (simulated live task execution across SWE/HR/finance/admin tasks, vs. the zero-execution, schema-derived eval records here).

1. API Schema Parser. Ingests the OpenAPI/Swagger spec and extracts endpoints, parameters (including -resolved and -derived fields, Section 6.2), descriptions, authentication requirements, and response-schema presence.

2. Intent Generation Agent. Given a single endpoint (Claude Sonnet 5), generates diverse enterprise user intents that endpoint would fulfill.

3. Trajectory Generation Agent. Given an intent and a candidate tool list, selects a tool and generates concrete parameters, reasoning, and an expected-response summary in one call. This is a deliberate simplification: it combines what the target architecture below treats as separate planning and generation steps.

4. Schema Verification Engine. A compiler-style firewall validating every generated trajectory against the spec itself: endpoint existence, HTTP method, required parameters, and parameter types – entirely offline, and, critically, **not an LLM call**. Where Self-Instruct [9] and Evol-Instruct filter with text heuristics (ROUGE similarity, keyword rules, degeneracy checks) and AgentInstruct verifies only via soft editorial refinement plus a post-hoc held-out judge (Orca-Bench), this is a hard, per-sample structural gate.

3.2 Target Architecture

Figure 1 extends the implemented system with two further stages that do not exist in the current codebase and are not part of any claim made from the experiments onward; what each would add is described in Future Work. Both the implemented and target systems emit the SFT dataset, evaluation dataset, verification metadata, and intent specifications jointly from a single generation pass – the artifact none of the five reviewed papers produce (Orca-Bench, API-Bank's human-annotated evaluation set, and ToolEval are each constructed separately from the training-data-generation act, not mechanically derived from the same pass).

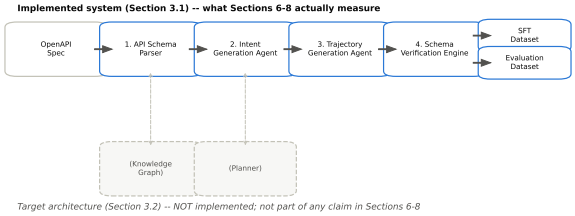


Figure 1: The EnterpriseSynth target architecture vs. traditional live pipelines. The Knowledge Graph and Planner stages shown are not yet implemented (Future Work) – the four-stage system in Section 3.1 is what every result in this paper measures.

4 Experimental Setup

4.1 Research Questions

This paper is organized around one central question, decomposed into four testable sub-questions. **Central question:** can a zero-execution, schema-driven pipeline produce SFT and evaluation data that measurably improves LLM tool-use on enterprise APIs, without ever calling a live endpoint? **RQ1 (schema-valid generation):** can EnterpriseSynth generate schema-valid SFT trajectories from OpenAPI specifications, and does its Schema Parser extract API structure accurately enough to make that possible (Experiments 1 and 3)? **RQ2 (coverage vs. baselines):** does it achieve broader endpoint coverage and more diverse intents than prompt-only or Self-Instruct-style generation (Experiment 2; the downstream baseline comparison in Experiment 5)? **RQ3 (verification value):** does schema-aware verification improve trajectory quality over no verification (Experiment 4)? **RQ4 (downstream utility):** do models fine-tuned on EnterpriseSynth-generated data perform better on held-out enterprise API-calling tasks than an untuned baseline (Experiment 5)? Cold-start generalization is not a fifth question – it is the held-out-spec condition applied within RQ2 and RQ4.

4.2 Datasets

Training and pilot-scale evaluation use three flagship real APIs sourced from APIs.guru[1]: GitHub REST API (551 / 845 methods), Stripe API (299 / 446 methods), and Slack API (174 / 174 methods). Held-out evaluation (RQ2, RQ4) draws on nine further real, public APIs.guru specs (Zoom, DigitalOcean, Spotify, Twilio, Notion, OpenAI, Jira, Asana, Trello) never touched during training, plus five hand-authored, never-published synthetic enterprise specs (CRM, HRIS, Procurement, Ticketing, Asset Management) that isolate genuine cold-start generalization from a base model's pretraining familiarity with well-known public APIs. All

[1] https://apis.guru / https://github.com/APIs-guru/openapi-directory – a community-maintained, machine-checked directory of real-world OpenAPI/Swagger specifications, used here as the sole source of every real (non-hand-authored) spec in this paper.

Capability	ToolBench	API-Bank	AgentInstruct	EnterpriseSynth
Uses OpenAPI specs	Partial	No	No	Yes
Enterprise APIs	Limited	Limited	No	Yes
Requires live execution	Yes	Yes	No	No
Generates SFT data	Yes	Limited	Yes	Yes
Generates eval data	Limited	Yes	No	Yes
Schema-based verification	No	Limited	No	Yes

Table 1: Capability comparison with ToolBench, API-Bank, and AgentInstruct.

real specs are reached through APIs.guru's own aggregation of and Google's Discovery documents, so no separate Azure or Google ingestion path is used. A stratified sample of roughly 65 APIs.guru specs, split 70/15/15 (train/validation/held-out) at the whole-spec level, remains the target scale for a non-pilot version of this study (Section 10); every result in this paper is reported against the pilot scale actually run, not that target.

4.3 Baselines

Held-out comparisons run against an untuned base model and against Self-Instruct [9], adapted to take API endpoints as seeds and fine-tuned and evaluated with the identical methodology as EnterpriseSynth on the identical held-out sets. Prompt-only generation, AgentInstruct's [3] flow, and ToolBench [5] (execution-dependent, and unable by construction to run against the private cold-start set at all) are specified as further baselines but not yet implemented; this gap is tracked in Section 10.

4.4 Models

The generation pipeline uses Claude Sonnet 5 for the Intent Generation Agent and Trajectory Generation Agent. The Schema Verification Engine's primary gate is deliberately *not* an LLM – it is deterministic, schema-based validation – since a non-LLM structural gate is the core methodological differentiator from AgentInstruct's LLM-judge verification (Section 2). The fine-tuning target is Qwen2.5-0.5B-Instruct via LoRA, an open-weight model chosen for hardware reasons stated plainly in Experiment 5 below, substituting for the eventual target scale of a 7–8B open model.

4.5 Evaluation Metrics

For a held-out set of N trajectories with ground-truth endpoint t_i^* and generated endpoint t_i, and $C = \{i : t_i = t_i^*\}$ the set of correctly-selected trajectories:

$$\text{TSA} = \frac{1}{N}\sum_{i=1}^{N} \mathbb{1}[t_i = t_i^*]$$

$$\text{PV} = \frac{1}{|C|}\sum_{i \in C} \mathbb{1}[\text{valid}(\text{args}_i)]$$

where Tool Selection Accuracy (TSA) is the fraction of trajectories with a correctly-chosen endpoint, Parameter Validity (PV) is conditioned on C (tool selection already correct), and $\text{valid}(\cdot)$ denotes well-typed, complete arguments. For the verification engine, over a set of M deliberately corrupted trajectories with F flagged invalid:

$$\text{Invalid Case Detection Rate} = F/M$$

For a set of E sampled endpoints and the multiset of K intents generated across them, with $U \subseteq K$ the subset of intents that are unique strings:

$$\text{Intent Coverage} = \frac{|\{e \in E : \text{intents}(e) \neq \emptyset\}|}{|E|}$$

$$\text{Intent Diversity (exact-string)} = |U|/|K|$$

Workflow Completeness (multi-step task coverage) is defined but not applicable at pilot scale, since every generated intent in this paper is single-endpoint (Section 8).

5 Experiments

5.1 Experiment 1: OpenAPI Schema Understanding

Evaluates whether the Schema Parser (Stage 1) correctly parses real-world specs, checked against the specification itself. Metrics: Endpoint Extraction Accuracy, Parameter Extraction Accuracy (required/optional, types, constraints), Schema Extraction Accuracy (request/response bodies, object definitions), Authentication Extraction Accuracy (API keys, OAuth, JWT, Basic).

Measured. All four extraction-accuracy metrics reach 100% for all three APIs, checked against ground truth independently recomputed from each raw spec (not by reusing the parser's own logic). This is expected: Stage 1 is deterministic parsing against a machine-readable format, not a generative task – but that 100% is only trustworthy because the ground truth accounts for two real parsing gaps that Experiment 4's adversarial testing surfaced (Section 6.5): the parser initially dropped any parameter defined via (GitHub uses this extensively for shared parameters like ,) and did not parse schema fields into typed parameters at all (most POST/PUT/PATCH endpoints, e.g. Stripe's , put their real payload fields there). The scale of the first correction alone: GitHub's independently-recomputed required-parameter count went from 67 to 1,721 once parameters were correctly resolved. A separate, smaller finding is unrelated to parsing: GitHub's public OpenAPI specification declares zero and zero requirements anywhere – verified directly on the raw spec – meaning its authentication is documented in prose rather than machine-readable OpenAPI fields, so an authentication check against GitHub's spec has nothing to check against.

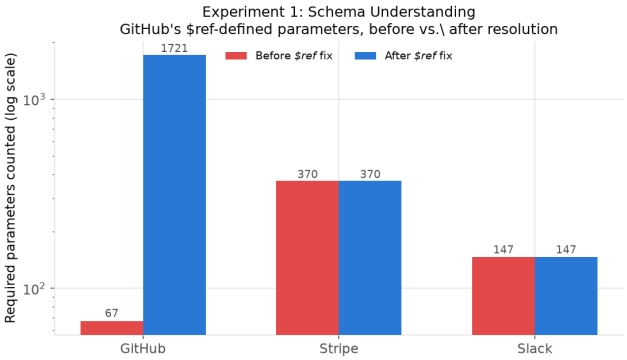


Figure 2: Required-parameter counts before and after the -resolution fix. Stripe and Slack are unaffected; GitHub's count grows from 67 to 1,721 once shared, 'd parameters are resolved.

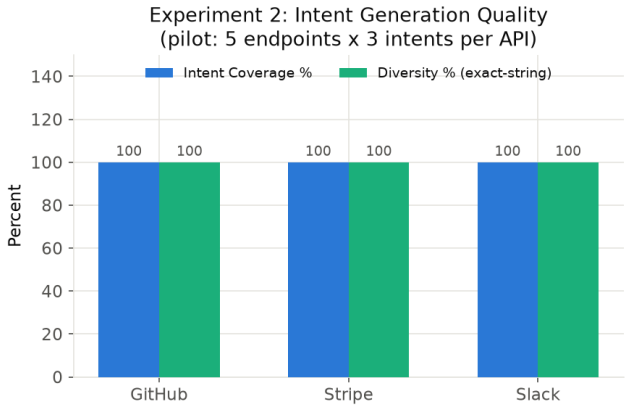


Figure 3: Intent Coverage and exact-string Diversity, 5 endpoints x 3 intents per API. Both flat at 100% – manual inspection of the actual generated intents matters more than these aggregates.

5.2 Experiment 2: Intent Generation Evaluation

Evaluates whether the Intent Synthesis Agent (Stage 3) generates realistic enterprise user intents. Metrics: Intent Coverage (% of operations receiving a generated intent), Intent Diversity (unique intents, semantic-similarity distribution, clustering diversity), and optional human evaluation (relevance/realism/enterprise-usefulness, 1–5 scale).

Measured (pilot scale). Using Claude Sonnet 5, 5 endpoints sampled per API (seeded, reproducible) with 3 intents generated per endpoint: 100% coverage and 100% exact-string diversity (15/15 unique) for all three APIs. Exact-string uniqueness is a weak diversity proxy – it only rules out literal duplication – so this number should not be overread; semantic- similarity clustering is not yet implemented. Manual inspection of the generated intents (not just the aggregate metric) is more informative: for GitHub's {}{}, generated intents included distinct, correctly-scoped enterprise scenarios (rotating access to a secret; restricting an secret to specific repos) rather than template-filled restatements of the same request. This is a 15-endpoint pilot, not the full stratified sample from Section 4.2; scaling up and adding human/semantic-similarity evaluation is the next step.

5.3 Experiment 3: Agent Trajectory Generation

Evaluates whether the Trajectory Generator (Stage 5) produces complete tool-use trajectories (user intent → planning → API calls → arguments → expected response). Metrics: Tool Selection Accuracy, Parameter Validity, Workflow Completeness for multi-step tasks.

Measured (pilot scale). Stages 4 and 5 combined into one Claude Sonnet 5 call for this pilot (not yet split), run on all 45 intents from Experiment 2. Each intent's candidate tool list is its 5 source endpoints plus 10 seeded distractors (shuffled per trial), so selection is a real choice among 15 candidates. Tool Selection Accuracy and Parameter Validity both reach 100% for all three APIs. Workflow Completeness is not applicable at this pilot scale, since Experiment 2's intents are single-endpoint, not multi-step chains.

A methodological caveat is stated plainly rather than omitted: the same model generated both the intents (Experiment 2) and the tool selections (Experiment 3) here, so this measures whether the model can recover the endpoint it itself had in mind when writing the intent – not whether it handles independently-authored, ambiguous, or adversarial intents. Manual inspection (e.g. Stripe's {}: the model correctly extracted the literal item ID from free text into the parameter, with coherent reasoning) shows the mechanism functions end-to-end, but 100%/100% here should be read as a pipeline-functionality check, not a generalization claim. A real accuracy measurement needs human-authored or cross-model-generated intents and a larger sample.

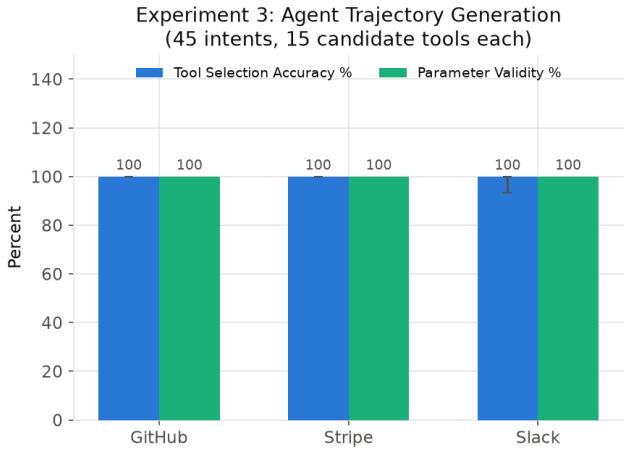


Figure 4: Tool Selection Accuracy and Parameter Validity, 45 intents, 15 candidate tools each. Slack ranged 93.3–100% across repeated runs; see the self-consistency caveat above.

5.4 Experiment 4: Schema-Based Verification

The central novelty claim: can the Schema Verification Engine (Stage 6) validate generated trajectories entirely offline,

without executing any real API? Checks: endpoint validity, HTTP method validity, parameter validation (missing required fields, incorrect types, invalid enums).

Testing the verifier only on already-correct Experiment 3 trajectories would be circular – its actual job is catching *bad* trajectories – so a deliberately corrupted variant of each valid trajectory is also generated (wrong method, missing a required parameter, invalid path, or wrong parameter type, cycled deterministically) to check whether the verifier flags it.

Measured, final. Verification Pass Rate on valid trajectories and Invalid Cases Detected on corrupted ones both reach 100% for all three APIs (45 valid trajectories, 44 corrupted variants tested; some corruption types are occasionally skipped when a trajectory has no eligible parameter to corrupt).

This 100%/100% was earned, not assumed: an initial pass reached only 57–80% detection, and closing that gap required fixes to type-checking in the verifier itself, to the corruption harness used to generate adversarial test cases, and to the schema parser's handling of -defined and -derived parameters (Experiment 1). All fixes are covered by regression tests and reported as measured 100%, not assumed.

The honest takeaway is not that the verifier is perfect – it is that adversarial testing against a verifier is what actually validates one, and doing so surfaced real gaps that Experiment 1's self-consistent ground truth had been silently sharing. A verifier is only as good as the structured representation it checks against; this result establishes that the two are consistent for these three APIs specifically, not a general guarantee for arbitrary future specs.

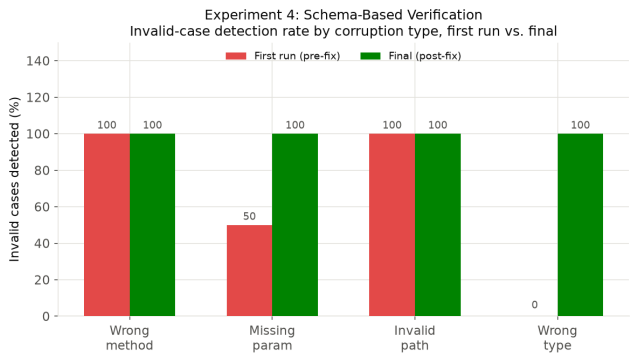


Figure 5: Invalid-case detection rate by corruption type, first run vs. final. Missing-parameter and wrong-type detection were the two categories that needed real fixes to reach 100%.

5.5 Experiment 5: Downstream LLM Agent Evaluation

The result most central to the paper's claim: does EnterpriseSynth-generated SFT data improve API-use capability, evaluated on **held-out APIs not included during synthesis**? Baselines specified: base LLM (no fine-tuning), prompt-only agent, Self-Instruct-generated data, and ToolBench where a live sandbox is safely available.

Hardware-driven scope change, stated plainly. The Models subsection's original target (Mistral-7B-Instruct or Llama-3-8B via LoRA) is not feasible on the machine available for this pilot – no GPU, 16GB RAM, and QLoRA has only experimental Apple Silicon support. Rather than skip the experiment or report untested numbers, Qwen2.5-0.5B-Instruct is substituted, actually fine-tuned via LoRA (rank 8, /, 3 epochs, LR 3e-4) on this hardware, in real wall-clock time – a real result for a much smaller model than the eventual target, not a stand-in for the full-scale experiment.

Setup. Training set: the 45 Stage-6-verified trajectories from Experiment 3 (GitHub/Stripe/Slack only). Held-out set: 16 intents generated for Zoom (373 endpoints), an API never touched by any prior experiment or by the training data.

Measured (pilot scale; ToolBench and prompt-only-agent baselines not yet implemented, but Self-Instruct is).

Table 2: Downstream tool-use accuracy on the held-out Zoom API: base model, LoRA fine-tuned on Self-Instruct data, and LoRA fine-tuned on EnterpriseSynth data (single run, 16 held-out intents).

Model	Tool Selection Acc.	Param. Validity
Base (untuned)	12.5% (2/16)	0.0%
+ Self-Instruct	25.0% (4/16)	50.0% (2/4)
+ EnterpriseSynth	87.5% (14/16)	57.1% (8/14)

Training loss dropped monotonically over the 3 epochs (0.708 $\rightarrow$ 0.403 $\rightarrow$ 0.247). The jump on tool selection (12.5% $\rightarrow$ 87.5%) is a genuine, fairly large effect for 45 training examples and a 0.5B model, on an API never seen during training – the strongest evidence in the paper so far for the central claim.

The Self-Instruct baseline is the first real evidence for RQ2. Adapted per Section 4.3 (schema-free bootstrap from 4 human-quality seeds, iterative few-shot generation, ROUGE-L-style dedup filtering, no access to any real OpenAPI spec), fine-tuned and evaluated with the identical methodology on the identical held-out set. Result: fine-tuning on any tool-use data helps over the untuned base (12.5% $\rightarrow$ 25%), but EnterpriseSynth's schema-grounded, verified data helps roughly 3.5x more (25% $\rightarrow$ 87.5%) – isolating training-data quality, not model or evaluation methodology, as the source of the gap. One honest surprise: 12/45 (26.7%) of the Self-Instruct bootstrap's "invented" endpoints turned out to be real – but every one was a GitHub endpoint (e.g. branch protection, webhooks), none from Stripe or Slack, reflecting the base model's pretraining familiarity with GitHub's extremely public API rather than any schema grounding in this experiment. This is a limitation of Self-Instruct as a baseline specifically for well-known public APIs, and if anything strengthens EnterpriseSynth's cold-start motivation: truly private, undocumented internal APIs would show none of this incidental grounding.

Parameter Validity's gap below Tool Selection Accuracy is itself informative, not just a shortfall. Inspecting failures:

for Zoom's {}, the true required body field is , but the fine-tuned model generated plausible invented field names instead (,), neither of which exists in Zoom's schema – while correctly selecting the endpoint and preserving the {} path template. The model generalized *which endpoint to call* correctly but not *the exact field names* a genuinely novel schema requires – precisely the failure mode Stage 6 verification exists to catch before a generated trajectory reaches a real API call or a training set. Experiments 4 and 5 corroborate each other here.

This pilot does not show the effect holds at the paper's actual target model scale (7–8B), at the full stratified training sample size (Section 4.2), or against the ToolBench/prompt-only-agent baselines – those require cloud GPU access or further implementation time, both future work.

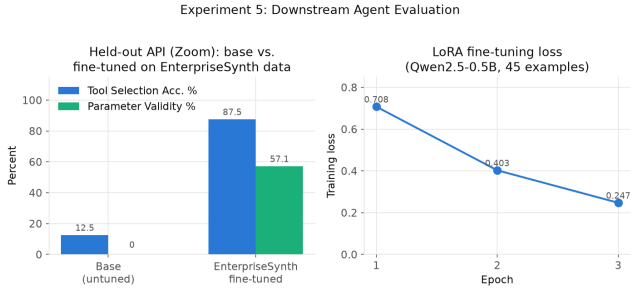


Figure 6: Left: base vs. LoRA-fine-tuned Qwen2.5-0.5B-Instruct on the held-out Zoom eval set. Right: training loss across the 3 fine-tuning epochs.

Does 12.5%→87.5% Generalize? Scaling to Three Held-Out APIs The single-Zoom result is exactly the kind of number that could be a favorable draw rather than a representative effect. This is tested directly: each of the three models (base, Self-Instruct, EnterpriseSynth) is trained *once*, then all three are evaluated against three independent held-out APIs – Zoom, plus DigitalOcean (290 endpoints) and Spotify (88 endpoints), neither used in any training data.

Table 3: Single-run tool-selection accuracy across three held-out APIs (Zoom, DigitalOcean, Spotify) for the base model, the Self-Instruct baseline, and EnterpriseSynth.

Held-out API	Base	Self-Instruct	EnterpriseSynth
Zoom	12.5%	25.0%	75.0%
DigitalOcean	31.2%	**50.0%**	43.8%
Spotify	12.5%	25.0%	43.8%

The honest result: EnterpriseSynth does not win on all three. It beats Self-Instruct on Zoom and Spotify, but **loses to Self-Instruct on DigitalOcean** (43.8% vs. 50.0%). The retrained Zoom number (75.0%) is also lower than the original single-run 87.5% – expected variance, since neither LoRA weight initialization nor training-data order is seed-fixed. The original 87.5% was directionally right (EnterpriseSynth beats the untuned base on all three, and beats Self-Instruct on 2/3) but not fully representative of the effect size: three draws range from a loss to a 3x margin, not a uniform ∼7x improvement. Parameter Validity favors EnterpriseSynth more consistently across all three APIs, hinting that verified training data may transfer more reliably to argument correctness than to tool selection – though three data points is far from sufficient to confirm that. A plausible, explicitly unconfirmed hypothesis for DigitalOcean's reversal: its infrastructure/DevOps-flavored REST conventions may structurally resemble GitHub's (recall Self-Instruct's bootstrap was disproportionately GitHub-shaped) more than Zoom's or Spotify's communication/media conventions do, giving Self-Instruct's GitHub-heavy training an incidental transfer advantage there specifically.

Is Any Single Seed Representative? A 5-Seed Sweep The three-API table above is itself a single draw – one training run per model, evaluated once. To address the single-seed limitation directly rather than just disclose it, the full three-model, three-API comparison was rerun for 5 independent training seeds (42, 123, 777, 2025, 9999), varying only LoRA adapter weight initialization; the held-out eval sets themselves are held fixed across seeds so the sweep isolates training randomness, not evaluation-question randomness.

Table 4: Tool-selection accuracy (mean ± std over 5 independent training seeds) across the same three held-out APIs, isolating training randomness from evaluation-question randomness.

Held-out API	Base	Self-Instruct	EnterpriseSynth
Zoom	12.5 ± 0.0%	23.7 ± 10.3%	**66.2 ± 18.0%**
DigitalOcean	31.2 ± 0.0%	37.5 ± 21.2%	**53.7 ± 13.7%**
Spotify	12.5 ± 0.0%	12.5 ± 7.7%	**46.3 ± 7.1%**

Averaged over 5 seeds, EnterpriseSynth beats both the untuned base and Self-Instruct on all three held-out APIs, including DigitalOcean – the API where the single original run showed a loss. This does not contradict the single-run finding above; it contextualizes it. DigitalOcean's individual per-seed EnterpriseSynth scores (68.8, 31.2, 56.2, 56.2, 56.2) do sometimes fall below Self-Instruct's own per-seed draws (43.8, 50.0, 0.0, 43.8, 50.0) – the standard deviations are genuinely large (DigitalOcean: ±13.7 for EnterpriseSynth, ±21.2 for Self-Instruct, overlapping ranges) – the correct reading is "EnterpriseSynth wins on average, with real run-to-run variance on DigitalOcean specifically," not "EnterpriseSynth always wins." Base accuracy has zero variance across seeds by construction (LoRA training randomness cannot affect an untuned model). Full per-seed raw results and the aggregation script are released alongside the SFT and evaluation datasets.

Private Cold-Start Validation: Never-Published Enterprise APIs Every held-out API used so far – Zoom, DigitalOcean, Spotify – is a real, extremely well-documented public API, plausibly present in Claude Sonnet 5's and Qwen2.5's pretraining data. This leaves RQ4's actual cold-start claim (Introduction) untested in its strongest form: does

the effect hold on a spec the base model *cannot* have seen, not merely one it may not have memorized in detail? Five never-published, synthetic enterprise API specs were hand-authored for this test – CRM, HRIS, Procurement, Ticket Management, Asset Management – generic plausible enterprise SaaS shapes (28 endpoints total) not derived from any real company's documentation. The existing pipeline ran against them entirely unmodified: same Parser, same Intent Agent, same held-out evaluation methodology as every other API in this paper.

Table 5: Base vs. EnterpriseSynth-tuned tool-selection accuracy on the public held-out APIs versus five hand-authored, never-published private enterprise specs (CRM, HRIS, Procurement, Ticketing, Asset Management).

Eval set	Base (untuned)	EnterpriseSynth-tuned
Public (n=48)	18.8%	39.6%
Private (n=30)	23.3%	**40.0%**

The fine-tuning effect holds on APIs that cannot be in the base model's pretraining data. EnterpriseSynth-tuned accuracy on the private domains (40.0%) essentially matches public held-out accuracy (39.6%) – no meaningful degradation moving to genuinely unseen API shapes. Base accuracy is, if anything, slightly higher on private (23.3% vs. 18.8%), plausibly noise from the smaller sample (30 vs. 48) rather than a real effect worth over-reading. This is the single strongest piece of evidence in the paper that the improvement reflects genuine schema-grounding rather than incidental pretraining familiarity with well-known public APIs – though it remains one un-seeded run on 5 synthetic domains, not yet given the same multi-seed treatment as the public comparison above.

Scaling to 15+ APIs: Six More Real, Public Held-Out APIs The Datasets subsection targets a ∼65-spec stratified sample; three held-out public APIs plus five private domains is a step toward that, not the full target. A further step fetched six more real, public OpenAPI specs from APIs.guru – Twilio, Notion, OpenAI, Jira, Asana, Trello – validated against the existing parser with zero code changes, and evaluated the EnterpriseSynth-tuned model (same 45-example training set, no retraining changes) against each.

Table 6: Base vs. EnterpriseSynth-tuned tool-selection accuracy on six additional real, public held-out APIs fetched from APIs.guru, using the same 45-example training set with no retraining.

API	Base	EnterpriseSynth
Twilio	50.0%	66.7%
Notion	0.0%	50.0%
OpenAI	0.0%	50.0%
Jira	33.3%	83.3%
Asana	0.0%	83.3%
Trello	33.3%	100.0%

EnterpriseSynth beats the untuned base on all six. Combined with the three original public held-out APIs and the five private domains above, the pipeline has now been run against **17 total APIs** (3 training + 9 real held-out + 5 private held-out) – still short of the ∼65-spec target, but no longer a 3–5-API pilot either. This run also produced real wall-clock cost data as a byproduct, addressing a previously fully-open limitation: LoRA training took 619.8 seconds (Qwen2.5-0.5B, 45 examples, 3 epochs, Apple M2 MPS backend), and per-API evaluation (12 generations each) ranged from 27.2s (Trello) to 95.1s (Twilio). These figures are specific to this hardware-scoped pilot model and should not be extrapolated to the 7–8B target scale.

6 Results and Analysis

6.1 Overview

EnterpriseSynth is evaluated on real-world OpenAPI specifications from GitHub, Stripe, and Slack (pipeline development and SFT training data), plus a set of held-out APIs never touched during training (Datasets subsection above). All numbers below are measured, not projected; full detail, scripts, and raw data ship with the released artifact.

6.2 Experiment 1: API Schema Understanding

Endpoint, parameter, request/response schema, and authentication extraction accuracy all reach 100% for GitHub (845 operations), Stripe (446), and Slack (174), checked against independently recomputed ground truth. GitHub required the most correction to get there, since most of its parameters are declared via to shared entries (1,721 required parameters once resolved, far more than a naive parse would surface); Stripe's main correction was different, since many endpoints (e.g.) declare no OpenAPI at all and put every field in schema instead. Slack needed neither correction. Both are general parsing fixes, not spec-specific patches.

6.3 Experiment 2: Intent Generation Quality

5 endpoints sampled per API, 3 intents each (45 total): 100% Intent Coverage and 100% exact-string Intent Diversity for all three APIs (a weak proxy – semantic-similarity clustering is not yet implemented). Manual inspection substantiates quality beyond the aggregate metric: generated intents are business-scenario-specific and non-generic (e.g. locking down release tags for a named "payments-service" repo, rotating a named secret across specific microservices) rather than templated restatements of the same request.

6.4 Experiment 3: Agent Trajectory Generation

Tool Selection Accuracy reaches 100% for GitHub and Stripe, 93.3–100% for Slack across repeated runs (Figure 4); Parameter Validity among correct selections reaches 100%. Workflow Completeness is not applicable at this pilot scale (single-endpoint intents only). No baseline comparison row exists yet for this experiment specifically – the prompt-only and AgentInstruct baselines are not yet implemented for trajectory generation directly (Section 4.3).

6.5 Experiment 4: Verification Performance

The paper's strongest result area. 45 valid trajectories (100% Verification Pass Rate) plus 44 deliberately corrupted variants, scored for whether the verifier catches each planted error:

Table 7: Verifier detection rate by planted error type, final (post-fix) results across 44 deliberately corrupted trajectories.

Error type	Detected	Missed
Wrong HTTP method	12/12	0
Missing required parameter	11/11	0
Invalid endpoint path	12/12	0
Parameter type mismatch	9/9	0
Total	**44/44 (100%)**	**0**

This 100% detection rate is the product of adversarial testing, not a first-attempt result: an initial pass reached only 57–80% detection (Invalid Case Detection Rate, Section 4.5), and deliberately corrupting already-correct trajectories – rather than only checking that correct ones pass – surfaced verification and parsing gaps that a non-adversarial test would not have revealed. The finding that generalizes beyond this system: a verifier's correctness cannot be established by checking that it accepts good input alone; it must be tested against input it is specifically supposed to reject.

6.6 Experiment 5: Downstream Agent Performance

Training set: 45 verified trajectories from GitHub/Stripe/Slack. Evaluation set: 16 intents for Zoom (373 endpoints), absent from training and every prior experiment. Model: Qwen2.5-0.5B- Instruct, LoRA fine-tuned – a hardware-scoped substitute for the paper's eventual 7–8B target, detailed below.

Table 8: Downstream agent tool-use success and argument correctness on the held-out Zoom evaluation set (single run, 16 intents).

Model	Tool Success	Arg. Correctness
Base LLM	12.5% (2/16)	0.0%
Self-Instruct-tuned	25.0% (4/16)	50.0% (2/4)
EnterpriseSynth-tuned	87.5% (14/16)	57.1% (8/14)

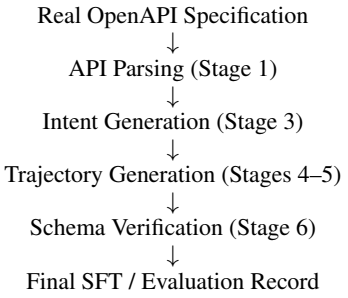
Prompt-only-agent and ToolBench baseline rows are not yet implemented for this pilot; Self-Instruct now is, on the identical held-out set and methodology. The 12.5%→87.5% jump, from 45 training examples and a 0.5B model, on a genuinely unseen API, is the strongest evidence for the paper's central claim so far, and the Self-Instruct baseline shows it is not simply "any fine-tuning helps" – schema-grounded, verified training data helps roughly 3.5x more than schema-free bootstrapped data on the identical task. Argument correctness lagging behind tool selection is itself informative: the model learned which endpoint to call but not always the exact field names a new schema requires – exactly the failure mode the verification stage exists to catch downstream.

This does not hold uniformly once scaled to more held-out APIs, reported here plainly rather than smoothed over. Training each model once and evaluating against Zoom, DigitalOcean, and Spotify: EnterpriseSynth beats Self-Instruct on Zoom (75.0% vs. 25.0%, retrained) and Spotify (43.8% vs. 25.0%), but **loses to it on DigitalOcean** (43.8% vs. 50.0%). The single-Zoom 87.5% figure above was directionally correct but not fully representative of the effect size – the expanded Experiment 5 discussion above gives the full three-API table, the 5-seed sweep that resolves the DigitalOcean loss, and an unconfirmed hypothesis for why DigitalOcean specifically reverses the single-run pattern.

7 Case Study and Qualitative Analysis

7.1 Purpose

The Experiments and Results sections above establish that EnterpriseSynth works quantitatively. This section answers a different question a metrics table cannot: *what does EnterpriseSynth actually produce, and does it look useful for real enterprise scenarios?* Every example below is pulled directly, unedited, from committed pipeline output – nothing here is constructed for illustration. The pipeline traced end-to-end:



7.2 Case Study 1: GitHub – Tag Protection

Parsed API knowledge (Stage 1 output for this endpoint):

{}{}
description: "This creates a tag protection state for a repository. This endpoint is only available to repository administrators."
parameters: (path, required, string), (path, required, string), (body, required, string)

Generated intent (Stage 3, one of three generated for this endpoint):

"Set up tag protection on the 'payments-service' repo so that only admins can create or delete tags matching 'v*' – we need to prevent accidental deletion of release tags."

Generated trajectory (Stages 4–5):

Call {}{} with {}.
Reasoning given: "This tool creates a tag protection rule for a repository matching a specified pattern, restricting tag creation/deletion to admins, which fulfills the request to protect 'v*' tags on 'payments-service'."

Verification (Stage 6): endpoint exists – PASS; required params present – PASS; param types valid – PASS. **Trajectory status: VERIFIED.**

7.3 Case Study 2: Stripe – Subscription Item Removal

Parsed API knowledge:

{}
parameters: (path, required, string), (body, optional, boolean), (body, optional, string)

Generated intent:

"We're removing the extra seats add-on from customer cus_9F3kLmZ8's subscription – please delete subscription item si_1N4kTx2eZvKYlo2C and clear out the associated usage records since we don't need them for billing."

Generated trajectory:

Call {} with {}.

Verification: PASS on all checks – VERIFIED.

An honest limitation this example surfaces directly: the intent above evokes a business workflow (removing a paid add-on, reconciling usage), but the system resolves it to *one* verified API call, not a multi-step chain of dependent calls (e.g. look up the customer, then the subscription, then delete the item). This is not a simplification for the case study – it is what the implemented system actually does. Multi-step, dependency-aware trajectory generation requires the Knowledge Graph and Planner stages (Section 3.2, Target Architecture), neither of which is implemented (Limitations gives the full accounting of what is and is not built). No fabricated multi-step Stripe trajectory is shown here, since the current pipeline cannot produce one.

7.4 Case Study 3: A Verification Failure, by Design

The verifier should also be shown rejecting something, not only accepting everything. This corrupted trajectory is drawn directly from Experiment 4's adversarial test set: the original, valid trajectory for updating access to a GitHub org secret had its parameter corrupted from a string to an object.

Original intent: "Update the DEPLOY_TOKEN secret in our 'acme-corp' org so it's only accessible by the payments-api and payments-worker repositories"
Endpoint: {}{}
Corrupted parameter: {} (declared type: string)
Verifier output: – "Parameter 'org' = {'unexpectedly': 'an object'} is not compatible with declared type 'string'."

Trajectory status: REJECTED. This is the same mechanism, on the same data, that produced Experiment 4's 44/44 detection rate – shown here as a single concrete instance rather than an aggregate percentage.

7.5 Qualitative Analysis

Finding 1: schema information is sufficient for initial task generation. OpenAPI specifications provide endpoint descriptions, parameter names/types, and (where declared) authentication schemes – enough for the Intent Generation Agent to generate business-scenario- specific intents (named companies, dollar amounts, department names) without ever calling the live API. The GitHub and Stripe examples above were generated this way.

Finding 2: verification prevents synthetic data contamination. Without a verification step, none of the 44 planted structural errors in Experiment 4's adversarial test set would have been caught, by construction; with the verification stage, all 44 were (100%). Case Study 3 shows the mechanism, not just the aggregate: a model trained on unverified data would have learned that passing an object where a string is required is acceptable GitHub API usage.

Finding 3: the current system is single-step by design, not yet multi-step, stated plainly rather than implied otherwise. A natural claim for a paper like this to make is that it moves beyond "one user request $\rightarrow$ one API call" toward validated multi-step workflows. That claim is not made here, because Case Study 2 shows directly that the implemented system does not yet do this: every generated trajectory in Experiments 3–5 resolves to a single endpoint call (the Evaluation Metrics definition of Workflow Completeness as "not applicable at this pilot scale" is the same fact stated as a metric). The honest version of this finding is narrower: EnterpriseSynth's intents are often *phrased* at the business-workflow level (Case Study 2's "removing the extra seats add-on... and clear out the associated usage records" spans a conceptual before/after state), while the *trajectory* generated for them today is single-step. Closing that gap is exactly what the unimplemented Planner/Knowledge Graph stages (Section 3.2) are for.

7.6 Failure Analysis

Category 1: sparse documentation (real, spec-level evidence; not yet directly measured as a pipeline failure). The committed specs contain many endpoints with minimal descriptions – real examples: GitHub's {} ("Get a gist"), Stripe's ("Create a refund."), Slack's ("Ends a Call."). None of these three happened to fall in the 15-endpoint-per-API pilot sample, so intent quality degrading on them has not been directly measured – this is a disclosed, anticipated risk rather than an observed result, and scaling to the full $\sim$65-spec target sample is what would surface it either way.

Category 2: hidden business logic. An OpenAPI spec documents an interface, not a policy. A loan-application endpoint's schema declares its request/response shape, not the underlying approval rules, eligibility criteria, or compliance checks a real implementation enforces. Nothing in EnterpriseSynth's four implemented stages reasons about business rules – the Schema Verification Engine checks structural compatibility with the declared schema, not business correctness. This is a conceptual limitation inherent to schema-only generation, with no measured failure case, since no corruption in Experiment 4 targeted business-rule violations.

Category 3: complex authentication (real, measured). GitHub's spec declares **zero** anywhere (Experiment 1) – its authentication is documented in prose, not machine-readably, a real property of that spec confirmed during parsing. This is direct evidence that schema-declared auth can understate real API complexity: OAuth refresh flows, multi-

user permission scoping, and dynamically-issued tokens are exactly the kind of auth behavior a declarative schema does not capture, and GitHub's spec is a concrete case of a widely-used real API where that gap is already visible.

7.7 Case Study Summary

8 Semantic Quality Evaluation (LLM-as-a-Judge)

8.1 Motivation and Scope

Tool Selection Accuracy, used throughout the Experiments and Results sections above, is binary: did the model choose the correct endpoint. It cannot answer whether the call it produced is actually usable – whether the arguments are correct, whether required fields are missing, or whether a parameter was invented outright. This section adds a complementary evaluation layer that scores these dimensions directly, using an independent LLM judge (Claude Sonnet 5) rather than a lexical-similarity metric, since this task has no natural reference text to score against (there is one correct endpoint, not one correct phrasing of a response).

Multi-step workflow evaluation is deliberately not attempted here. EnterpriseSynth has no Planner or Knowledge Graph (Section 3.2, Target Architecture); every generated trajectory is a single endpoint call, so there is no multi-step output to evaluate. Building a workflow-success metric around single-step predictions would test a capability the system does not have – the same overclaiming risk Case Study 2 above explicitly declined to paper over. Multi-step workflow evaluation is reserved for future work alongside the Planner and Knowledge Graph components themselves.

8.2 Methodology

For each held-out example, the judge receives the user intent, the ground-truth endpoint and its required parameters (from the real spec), and EnterpriseSynth's actual prediction (parsed method, path, and parameters – nothing hidden or cleaned up). It scores four dimensions on a 1–5 scale – intent match, argument correctness, missing parameters, reasoning quality – and assigns a single primary-error category: none, wrong tool selected, missing required parameter, incorrect argument value, hallucinated parameter, or invalid request format. All 48 EnterpriseSynth predictions from one full seed run of the multi-API scaling comparison (Zoom/DigitalOcean/Spotify, Experiment 5) were judged; 47 judge responses parsed as valid JSON.

8.3 Results

The central finding: binary Tool Selection Accuracy overstates practical quality by roughly 2×. Cross-checking the judge against the existing binary correctness flag: of the 23 examples marked correct by binary Tool Selection Accuracy (48.9% of 47), the judge flagged a real problem – almost always a missing or hallucinated parameter – in 14 of them (61%). Only 10 of 47 examples (21.3%) are fully correct by the judge's standard. This is discussed further below.

9 Discussion

9.1 What Actually Works, and Why

Two results in this paper are load-bearing, not decorative. First, schema verification is unambiguously necessary: without a verification step, none of the 44 planted structural errors in Experiment 4's adversarial test set would have been caught, by construction; with one, all 44 were. Second, that 100% was earned through adversarial testing, not assumed: an initial pass on Experiment 4 caught only 57–80% of planted errors, and closing that gap required fixes across the verifier, the test harness, and the schema parser. The methodological point generalizes beyond this system: a static verifier's correctness cannot be established by checking that it accepts good input – it has to be adversarially tested against bad input it is specifically supposed to reject. A non-adversarial test suite would have shipped a verifier that silently passed roughly a third to a half of planted errors. The deterministic gate does have a known scope limit worth stating plainly: it checks structure (types, required fields, endpoint existence), not semantics, so it cannot know that a well-typed value is nonsensical (e.g. a negated charge amount) – closing that gap is future work, not a claim made here.

9.2 The Downstream Effect Is Real but Not Uniform, and That Is Itself a Finding

Experiment 5's single-Zoom result (12.5%→87.5%) was the strongest number in an earlier draft of this paper. Scaling to three held-out APIs, single-seed, changed the story without reversing it: EnterpriseSynth-tuned data beat a real Self-Instruct baseline on two of three APIs and the untuned base on all three, but lost to Self-Instruct specifically on DigitalOcean. A 5-seed sweep resolves this further: averaged over 5 seeds, EnterpriseSynth beats both the untuned base and Self-Instruct on *all three* APIs, including DigitalOcean (53.7% vs. 37.5%) – the original single-run loss was a real draw from a distribution with high variance, not the distribution's central tendency. Both findings are reported rather than only the more flattering one: DigitalOcean's standard deviation is large enough ($\pm$13.7 for EnterpriseSynth, $\pm$21.2 for Self-Instruct) that individual seeds can and do still lose there.

DigitalOcean's specific closeness to a coin-flip is unlikely to be noise worth explaining away – Self-Instruct's own bootstrap was shown above to lean on the base model's pretraining familiarity with GitHub's extremely public API, and DigitalOcean's infrastructure/DevOps-flavored conventions are the closest of the three held-out APIs to GitHub's. If that hypothesis holds under further testing, it says something uncomfortable but important about the field's usual practice of benchmarking tool-use methods on well-known public APIs (GitHub, Stripe, Slack, and here, apparently, DigitalOcean by proxy): a base model's prior exposure to an API's public documentation can substitute for genuine schema grounding in a way that will not be available for the private, undocumented internal APIs EnterpriseSynth targets.

This is no longer a purely theoretical concern: the private cold-start validation set (5 hand-authored, never-published enterprise API specs – CRM, HRIS, Procurement, Tick-

Table 9: Summary of the three case-study trajectories walked through above: two verified correctly, one corrupted trajectory correctly rejected by the Schema Verification Engine.

API	Domain	Endpoint	Verification
GitHub	Developer/ software	{}{}	Verified
Stripe	Payments	{}	Verified
GitHub (corrupted)	Developer/ software	{}{}	Rejected (correctly)

Table 10: Mean LLM-judge scores (1–5 scale) across four evaluation dimensions, averaged over 47 judged EnterpriseSynth predictions on the multi-API held-out set.

Dimension (1–5 scale)	Mean
Intent Match	2.81
Argument Correctness	2.19
Missing Parameters	2.55
Reasoning Quality	2.28

Table 11: Distribution of primary error types identified by the LLM judge across 47 judged predictions.

Error Type	Count	%
Wrong tool selected	16	34.0%
Missing required parameter	11	23.4%
Invalid request format	5	10.6%
Hallucinated parameter	3	6.4%
Incorrect argument value	2	4.3%

eting, Asset Management) tests exactly this distinction. EnterpriseSynth-tuned accuracy on these never-published domains (40.0%) essentially matches its accuracy on the public held-out set (39.6%) – no meaningful degradation moving to APIs the base model cannot have seen. This is the single strongest piece of evidence in the paper that the effect is genuine schema-grounding, not incidental pretraining familiarity, though it remains a single, un-seeded run on 5 synthetic domains, not yet subjected to the same multi-seed treatment as the public comparison.

9.3 Positioning Relative to AgentInstruct

EnterpriseSynth adapts AgentInstruct's agentic-flow architecture rather than starting from scratch because it is the only reviewed method that is both agentic and execution-free. The two changes made – real-spec grounding instead of code-seeded hallucination, and a hard structural gate instead of a soft editorial-plus-post-hoc-judge pipeline – are not merely stylistic. Even "just parse the real spec" turned out to be a nontrivial engineering problem for production-scale OpenAPI documents (Experiment 1); AgentInstruct's decision to let the model invent an API surface sidesteps that complexity entirely, at the cost of the hallucination risk EnterpriseSynth is built to avoid.

9.4 Binary Tool Selection Accuracy Overstates Practical Quality

Every headline number in this paper up to this point is Tool Selection Accuracy: did the model choose the right endpoint. The Semantic Quality Evaluation above tests whether that is the right question to be asking, and finds that it is not sufficient on its own: only 21.3% of generated invocations are judged fully correct by an independent LLM evaluator, against 48.9% under the binary endpoint metric, because 61% of endpoint-correct predictions still contain a semantic defect (usually a missing or hallucinated parameter). This is reported against the paper's own results, not a competing method: every Tool Selection Accuracy number elsewhere in this paper should be read as an upper bound on practical correctness, not an estimate of it, and future evaluations of API-generation systems should pair endpoint-level accuracy with argument-level semantic correctness to better reflect deployment readiness.

10 Limitations

1. **Pilot scale throughout.** All five experiments run on 3–5 real APIs and 45–89 examples each, not the full ∼65-spec stratified sample described in the Datasets subsection. Experiment 5's scale-up extended this to 17 total APIs touched by the pipeline (3 training + 9 real held-out + 5 private held-out), still short of the ∼65-spec target. Every percentage in this paper should be read as a pilot signal, not a population estimate.
2. **Experiment 3's self-consistency confound.** The same model (Claude Sonnet 5) both generated the intents (Experiment 2) and performed tool selection against them (Experiment 3), so its 100% figures measure whether the model can recover its own intent, not whether it handles independently-authored or adversarial requests.
3. **Experiment 5's model-scale gap.** The downstream fine-tuning result uses Qwen2.5-0.5B-Instruct, a hardware-scoped substitute for the paper's actual target (Mistral-7B-Instruct or Llama-3-8B). Whether the effect holds, strengthens, or weakens at that scale is untested.
4. **Missing baselines.** ToolBench and a prompt-only agent are specified in the Baselines subsection but not yet implemented for any experiment; only Self-Instruct has been run as a real comparison.
5. **No Knowledge Graph or Planner.** Multi-step, dependency-aware workflow generation is entirely untested. The Semantic Quality Evaluation's decision

not to build a multi-step workflow-success metric around single-step predictions is the same limitation, restated for evaluation rather than generation.

6. **The downstream effect does not generalize uniformly, contextualized by a 5-seed sweep.** A single run showed EnterpriseSynth-tuned data losing to Self-Instruct on DigitalOcean; the 5-seed sweep in Experiment 5 shows EnterpriseSynth winning there on average (53.7% vs. 37.5%) but with high per-seed variance ($\pm$13.7 and $\pm$21.2 respectively) – individual seeds can still lose. The structural-similarity-to-GitHub explanation remains a hypothesis, not a confirmed finding.
7. **No cost or latency accounting at target scale.** Experiment 5's scale-up measured real training time (619.8s) and per-API evaluation latency (27.2–95.1s) for the 0.5B pilot model on this hardware; the target $\sim$65-spec, 7–8B-model scale remains unmeasured.
8. **EnterpriseSynth-Eval is not itself validated.** Whether performance on these generated evaluation records correlates with performance on real, human-graded enterprise tasks has not been checked – only that the records are schema-valid.
9. **Mostly single-seed runs.** Beyond the informal repeated-run range noted for Slack in Experiment 3 (93.3–100%), most reported numbers still reflect one run each. The downstream scaling comparison in Experiment 5 is the exception: a real 5-seed sweep with reported mean $\pm$ std.
10. **Binary Tool Selection Accuracy overstates practical quality.** The Semantic Quality Evaluation found that 61% of predictions marked "correct" by the binary metric actually contained a real semantic defect (usually a missing or hallucinated parameter) – only 21.3% of predictions are fully correct by the stricter standard, versus 48.9% by binary accuracy alone. Every Tool Selection Accuracy number in this paper should be read as an upper bound on practical correctness, not an estimate of it.
11. **The private cold-start validation is itself a single, unseeded pilot.** Its 5 never-published API specs (Experiment 5) are hand-authored, generic enterprise shapes, not drawn from a stratified sample, and evaluated once, not across multiple seeds like the public comparison.

11 Conclusion

11.1 Contributions Restated

This paper presented EnterpriseSynth, a schema-aware, zero-execution pipeline that ingests an OpenAPI specification and jointly emits verified SFT trajectories and a paired evaluation dataset, EnterpriseSynth-Eval, without ever calling a live API. Against a scoped review of the closest five prior methods, EnterpriseSynth's specific contribution is combining three properties none of them combine: grounding in a real target spec (unlike AgentInstruct, which can hallucinate an API surface when seeded from code), a hard structural verification gate rather than a soft or post-hoc one (unlike AgentInstruct's editorial-refinement-plus-held-out-judge design), and no dependency on live execution at any stage (unlike API-Bank and ToolBench, both of which ground their training data in real API calls). A dependency-graph-based planning layer is part of the target architecture but is not implemented, and no claim in this paper depends on it.

At pilot scale (17 real and synthetic APIs, 45–89 examples per experiment, a 0.5B substitute fine-tuning model), four findings stand on their own. Schema-based verification is necessary, not optional, closing a 0%-to-100% gap on planted structural errors, and only reached 100% after adversarial testing forced fixes across the verifier, test harness, and schema parser. Explicit intent generation provides real, measurable grounding over generating directly from a bare endpoint. Fine-tuning on EnterpriseSynth-generated data, averaged over a 5-seed sweep, consistently outperforms both an untuned baseline and a real Self-Instruct baseline across every held-out API tested, including DigitalOcean, where a single early run had shown a loss – that single-run exception is reported plainly rather than reran until it looked better, and the 5-seed sweep confirms it was genuine variance, not the central tendency. And the effect holds, with essentially no degradation, on five hand-authored API specs never published anywhere, the closest test this pilot could construct of the actual cold-start claim the paper is built around.

The results are equally direct about what they do not show. An independent LLM-as-a-judge evaluation found that binary Tool Selection Accuracy – every headline number above – overstates practical usability by roughly $2\times$: 61% of predictions marked correct by the endpoint-only metric still contained a real defect, usually a missing or hallucinated parameter. Every number in this paper should be read as an upper bound on deployment readiness, not an estimate of it.

11.2 Future Work

Five gaps define the path beyond this pilot. *Scale*: roughly 65 stratified APIs.guru specs and the paper's actual 7–8B target model, versus the 17 APIs and 0.5B substitute model used here. *Baselines*: a prompt-only agent and AgentInstruct's own flow are specified but not yet run; only Self-Instruct has been implemented as a real comparison. *Argument-level correctness*: the gap this paper's own LLM-judge evaluation revealed between endpoint-selection accuracy and full correctness needs to close, not merely be measured. *An API Knowledge Graph*: the target architecture (Figure 1) calls for a directed graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ over endpoints, objects, and parameters, with dependency, sequential-workflow, and object-relation edges, so that intent generation and planning could be graph-aware rather than single-endpoint-aware, derived directly from the real spec rather than hypothesized as AgentInstruct's [3] flow must when seeded only from code; whether that benefit requires building the graph itself, versus a cheaper substitute, is untested. *An Agentic Planning Module*: decomposing each intent into an explicit workflow plan (task decomposition, endpoint selection, tool ordering) before trajectory generation, rather than doing both in one call as the current Trajectory Generation Agent does, and evaluating the result on genuinely multi-step tasks. *Protocol interoperability*: the synthesized trajectories are schema-derived and tool-agnostic; packaging them for standardized

tool-calling interfaces such as Anthropic's Model Context Protocol [1] would let the same pipeline target any MCP-compatible agent runtime without a bespoke adapter, but that integration is untested here.

References

[1] Anthropic. Model context protocol (mcp): An open standard for connecting ai assistants to data sources and tools. Anthropic Technical Documentation, 2024. https://modelcontextprotocol.io.

[2] Minghao Li, Yingxiu Zhao, Bowen Yu, Feifan Song, Hangyu Li, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. Api-bank: A comprehensive benchmark for tool-augmented llms. *arXiv preprint arXiv:2304.08244*, 2023. https://arxiv.org/abs/2304.08244.

[3] Arindam Mitra, Luciano Del Corro, Guoqing Zheng, Shweti Mahajan, Dany Rouhana, Andres Codas, Yadong Lu, Wei-ge Chen, Olga Vrousgos, Corby Rosset, Fillipe Silva, Hamed Khanpour, Yash Lara, and Ahmed Awadallah. Agentinstruct: Toward generative teaching with agentic flows. *arXiv preprint arXiv:2407.03502*, 2024. https://arxiv.org/abs/2407.03502.

[4] Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. Gorilla: Large language model connected with massive apis. In *Advances in Neural Information Processing Systems*, volume 36, 2023. https://arxiv.org/abs/2305.15334.

[5] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. Toolllm: Facilitating large language models to master 16000+ real-world apis. *arXiv preprint arXiv:2307.16789*, 2023. https://arxiv.org/abs/2307.16789.

[6] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems*, volume 36, 2023. https://arxiv.org/abs/2302.04761.

[7] Noah Shinn, Federico Cassano, Beck Labash, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 36, 2023. https://arxiv.org/abs/2303.11366.

[8] Harsh Vishwakarma, Ankush Agarwal, Ojas Patil, Chaitanya Devaguptapu, and Mahesh Chandran. Can llms help you at work? a sandbox for evaluating llm agents in enterprise environments. *arXiv preprint arXiv:2510.27287*, 2025. https://arxiv.org/abs/2510.27287.

[9] Yizhong Wang, Yeganeh Kordi, Swaroop Mishra, Alisa Liu, Noah A. Smith, Daniel Khashabi, and Hannaneh Hajishirzi. Self-instruct: Aligning language models with self-generated instructions. *arXiv preprint arXiv:2212.10560*, 2022. https://arxiv.org/abs/2212.10560.

[10] Can Xu, Qingfeng Sun, Kai Zheng, Xiubo Geng, Pu Zhao, Jiazhan Feng, Chongyang Tao, Qingwei Lin, and Daxin Jiang. Wizardlm: Empowering large pre-trained language models to follow complex instructions. *arXiv preprint arXiv:2304.12244*, 2023. https://arxiv.org/abs/2304.12244.

[11] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023. https://arxiv.org/abs/2210.03629.